

PICAPD INSTRUCTION SET ARCHITECTURE

Programmer's Reference Manual v1.0

Physics-Informed Computing for Autonomous Population Dynamics

1. ARCHITECTURAL STATE

1.1 Register Files

1.1.1 General-Purpose Registers (GPR)

Register Bank	Count	Width	Purpose
Integer (X)	32	64-bit	General computation, addressing
Floating-Point (F)	32	64-bit	FP64 arithmetic, moments
Event (E)	512	1-bit	Constraint states (0=violated, 1=satisfied)
Moment (M)	128	64-bit	Population moments μ_0 - μ_3 (32 sets $\times$ 4 moments)
Context (C)	64	1024-bit	Streaming context buffers
AGM State (A)	32	64-bit	Arithmetic-Geometric Mean iteration state
Vector Clock (V)	16	64-bit	Causal ordering timestamps

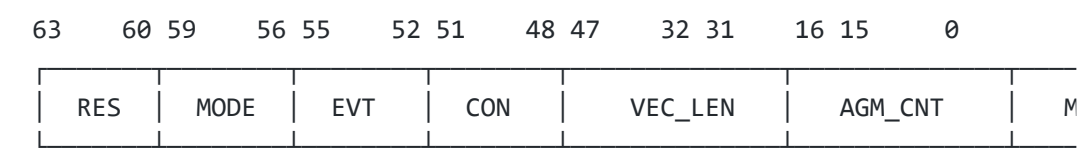
Register Bank	Count	Width	Purpose
Trust Score (T)	256	32-bit	Agent reputation metrics

Note: All registers are directly addressable by instructions. Event registers are bit-addressable.

1.1.2 Special Registers

Register	Width	Description
PC	64-bit	Program counter (instruction pointer)
STATUS	64-bit	Processor status and control
VECTOR	64-bit	Active vector length for SIMD operations
EPU_ID	16-bit	Event Processing Unit identifier (0-99 for workers, 100-109 for managers, 110 for queen)
HIERARCHY	16-bit	Current agent hierarchy level
TIMESTAMP	64-bit	Cycle-accurate timestamp counter

1.1.3 STATUS Register Format



MODE[3:0]: 0000=Hibernated, 0001=Active, 0010=Rollback, 0011=Wait
EVT[3:0]: Event propagation state
CON[3:0]: Constraint violation mask
VEC_LEN: Current vector length (0-65535)
AGM_CNT: AGM iteration counter (0-31)
MOM_ERR: Moment realizability error flags



1.2 Memory Address Space

1.2.1 Address Map

0x0000_0000_0000_0000 - 0x0000_0000_3FFF_FFFF: L0 Lagrangian State (51
0x0000_0001_0000_0000 - 0x0000_0001_3FFF_FFFF: L1 Trajectory Cache (1
0x0000_0002_0000_0000 - 0x0000_0002_FFFF_FFFF: L2 Global Shared (4 GB)
0x0000_0010_0000_0000 - 0x0000_001F_FFFF_FFFF: HBM3 Main Memory (64 GB
0xFFFF_0000_0000_0000 - 0xFFFF_0000_0000_FFFF: I/O Space (64 KB)
0xFFFF_8000_0000_0000 - 0xFFFF_8000_7FFF_FFFF: Configuration/CSR (2 GB



1.2.2 Byte Ordering

- **Little-endian:** All multi-byte values stored with least significant byte at lowest address
- **Alignment:** Natural alignment required (8-byte aligned for 64-bit, 16-byte for 128-bit)
- **Misalignment:** Generates alignment exception (trap code 4)

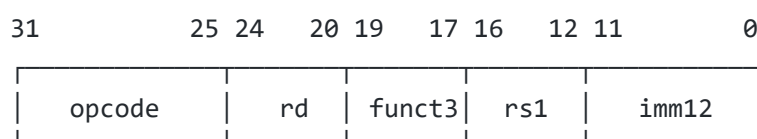
1.3 I/O Space

Memory-mapped I/O with separate 64KB address space. I/O registers are 32-bit wide.

2. INSTRUCTION FORMATS

2.1 Fixed 32-bit Formats

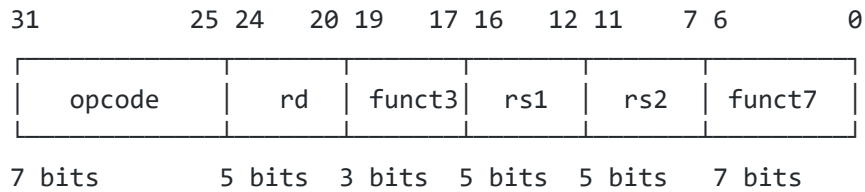
2.1.1 Type-E (Event Control)



7 bits 5 bits 3 bits 5 bits 12 bits

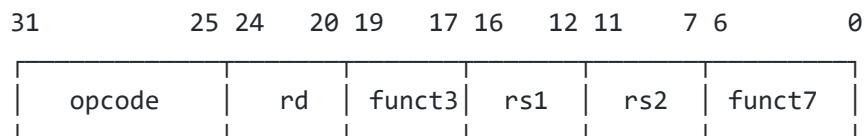
Purpose: Event register operations with 12-bit immediate

2.1.2 Type-M (Moment Operations)



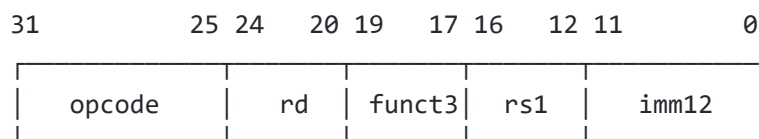
Purpose: Moment calculations with two source registers

2.1.3 Type-A (AGM Operations)



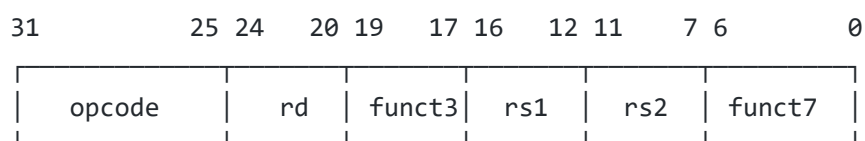
Purpose: Arithmetic-Geometric Mean iterations

2.1.4 Type-C (Context Flow)



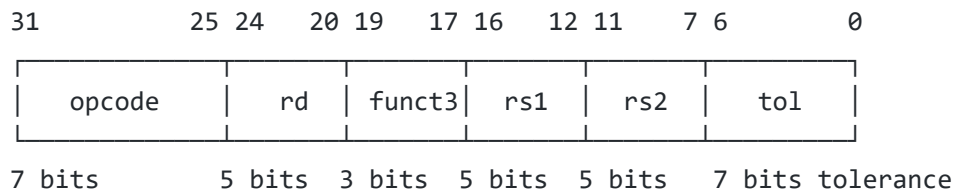
Purpose: Context streaming operations

2.1.5 Type-H (Hierarchy)



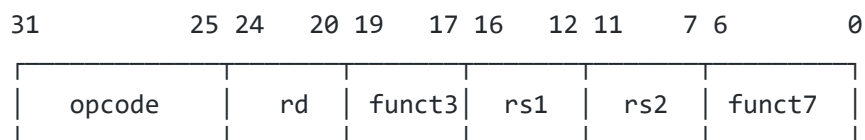
Purpose: Agent hierarchy management

2.1.6 Type-S (Safety/Constraint)



Purpose: Safety checking with tolerance specification

2.1.7 Type-V (Variational Mechanics)



Purpose: Variational mechanics operations

2.2 Immediate Formats

- **imm12**: 12-bit signed immediate, sign-extended to 64 bits
- **imm5**: 5-bit unsigned immediate for moment order k
- **tol**: 7-bit tolerance specification (0=strict to 127=disabled)

2.3 Opcode Allocation

Opcode [6:0]	Category	Base Instructions
0000011	Event Control	ESET, ECLEAR, ETEST, EWAIT, EBCAST
0001011	Moment Operations	MOM.CALC, MOM.COMP, MOM.XPORT, MOM.REAL

Opcode [6:0]	Category	Base Instructions
0010011	AGM/Elliptic	AGM.ITER, ELI.COMP, TXF.PASS
0011011	Context Flow	CTX.SLICE, CTX.AGG, CTX.SYNTH
0100011	Hierarchy	HIER.SPAWN, HIER.TERM, HIER.EVOL
0101011	Safety/Constraint	CONS.CHK, RES.BUDG, BYZ.CONS, TMR.VOTE, SAFE.ROLL
0110011	Inter-Agent Comm	MSG.SEND, MSG.RECV, MSG.ACK, CONS.PROP, WAL.LOG
0111011	Variational Mech	LEVAL, LGRAD_Q, LGRAD_V, ACTION, VERLET, HAMILT, AGRAD, ATHRESH
1000011	Coordinate Xform	CYCLIC, CTRANS, CJACOB
1001011	Sensor Fusion	SFSPU.SYNC, SFSPU.KF, SFSPU.IPS, SFSPU.PROJ
1110011	System/Privileged	ECALL, ERET, MRET, WFI, CSR*

3. ADDRESSING MODES

3.1 Register Addressing

- **Register-direct:** All operands in registers (Type-M, Type-A, Type-H, Type-V)
- **Register-indirect:** Base register + offset (load/store instructions)

3.2 Immediate Addressing

- **Immediate:** Constant embedded in instruction (Type-E, Type-C)
- **PC-relative:** PC + signed offset (branch instructions)

3.3 Memory Addressing

- **Base+offset:** Effective address = $X[rs1] + \text{sign-extend}(\text{imm12})$
- **Base+index:** Effective address = $X[rs1] + X[rs2]$ (for vector loads)
- **PC-relative:** For instruction fetch and constant pools

3.4 Event Addressing

- **Direct:** Event register indexed by 12-bit immediate (0-511)
- **Indirect:** Event register indexed by $X[rs1]$ (lower 9 bits used)

3.5 Context Addressing

- **Streaming:** Context registers indexed by 6-bit field (from $\text{imm12}[5:0]$)
- **Broadcast:** All context registers (special encoding)

4. INSTRUCTION SET

4.1 Event Control Instructions

ESET - Set Event Register

Format: ESET $rs1$, $imm12$
Encoding: $[0000011][rs1][000][rs1][imm12]$
Operation: $\text{event}[imm12[8:0]] \leftarrow 1$
 Broadcast wake signal to dependent EPU's
Flags: None
Exceptions: EventOutOfRange (if $imm12 > 511$)

ECLEAR - Clear Event Register

Format: ECLEAR $rs1$, $imm12$
Encoding: $[0000011][00000][001][rs1][imm12]$
Operation: $\text{event}[imm12[8:0]] \leftarrow 0$

Initiate rollback cascade for dependent computations
 Flags: STATUS.CON[ROLLBACK] $\leftarrow$ 1
 Exceptions: EventOutOfRange, RollbackRequired

EEST - Test Event State

Format: EEST rd, imm12
 Encoding: [0000011][rd][010][00000][imm12]
 Operation: $X[rd] \leftarrow \text{zero_extend}(\text{event}[\text{imm12}[8:0]])$
 Flags: None
 Exceptions: EventOutOfRange

EWAIT - Wait for Event

Format: EWAIT imm12
 Encoding: [0000011][00000][011][00000][imm12]
 Operation: Stall execution until $\text{event}[\text{imm12}[8:0]] == 1$
 Core enters low-power wait state
 Flags: STATUS.MODE $\leftarrow$ WAIT
 Exceptions: EventOutOfRange

EBCAST - Broadcast Event

Format: EBCAST imm12, rs1
 Encoding: [0000011][00000][100][rs1][imm12]
 Operation: Broadcast event[imm12[8:0]] state to EPU's in bitmask $X[rs1]$
 Flags: None
 Exceptions: EventOutOfRange



4.2 Moment Operations

MOM.CALC - Calculate Moment

Format: MOM.CALC rd, rs1, imm5
 Encoding: [0001011][rd][000][rs1][imm5][0000000]
 Operation: $F[rd] \leftarrow \int_0^\infty x^{\{\text{imm5}\}} n(x) dx$
 where $n(x)$ distribution starts at address $X[rs1]$

Flags: STATUS.MOM_ERR[PRECISION] on numerical error
Exceptions: MemoryFault, DivideByZero

MOM.COMP - Compress Moments

Format: MOM.COMP rd, rs1, rs2
Encoding: [0001011][rd][001][rs1][rs2][0000000]
Operation: $M[rd] \leftarrow \text{aggregate}(M[rs1], M[rs2])$ with 89.7:1 compression
Verifies Hausdorff inequality $\mu_1^2 \leq \mu_0 \cdot \mu_2$
Flags: STATUS.MOM_ERR[HAUSDORFF] if violated
Exceptions: ConstraintViolation



MOM.XPORT - Moment Transport

Format: MOM.XPORT rd, rs1, rs2
Encoding: [0001011][rd][010][rs1][rs2][0000000]
Operation: $M[rd] \leftarrow \text{evaluate } \partial\mu/\partial t + \nabla \cdot (u\mu) = S(\mu)$
where $u = F[rs1]$, source terms from $M[rs2]$
Flags: STATUS.MOM_ERR[CONSERVATION] on violation
Exceptions: ConstraintViolation

MOM.REAL - Realizability Check

Format: MOM.REAL rd, rs1
Encoding: [0001011][rd][011][rs1][00000][0000000]
Operation: $X[rd] \leftarrow \text{violation_bitmask}(M[rs1])$
Checks: $\mu_0 \geq 0$, $\mu_1^2 \leq \mu_0 \cdot \mu_2$, Hankel matrix ≥ 0
Flags: STATUS.MOM_ERR $\leftarrow X[rd]$
Exceptions: None

4.3 AGM Operations

AGM.ITER - Single AGM Iteration

Format: AGM.ITER rd, rs1, rs2
Encoding: [0010011][rd][000][rs1][rs2][0000000]
Operation: $a_{\text{new}} = (A[rs1].a + A[rs2].g)/2$

```

        g_new =  $\sqrt{A[rs1].a \times A[rs2].g}$ 
        A[rd]  $\leftarrow$  (a_new, g_new)
Flags:    STATUS.AGM_CNT  $\leftarrow$  STATUS.AGM_CNT + 1
Exceptions: None

```

ELI.COMP - Complete Elliptic Integral

```

Format:    ELI.COMP rd, rs1
Encoding:  [0010011][rd][001][rs1][00000][0000000]
Operation:  $F[rd] \leftarrow K(m) = \pi / [2 \cdot \text{AGM}(1, \sqrt{1-m})]$ 
           where  $m = F[rs1]$ 
           Executes 5 AGM iterations internally
Flags:     None
Exceptions: DomainError (if  $m \notin [0,1)$ )

```

TXF.PASS - Transfer Function Passivity

```

Format:    TXF.PASS rd, rs1, rs2
Encoding:  [0010011][rd][010][rs1][rs2][0000000]
Operation:  $X[rd] \leftarrow 1$  if all poles in LHP, else 0
           Numerator coeffs at M[rs1], denominator at M[rs2]
           Uses Routh-Hurwitz criterion
Flags:     None
Exceptions: None

```

4.4 Context Flow Operations

CTX.SLICE - Context Slicing

```

Format:    CTX.SLICE rd, rs1
Encoding:  [0011011][rd][000][rs1][00000][0000000]
Operation: Partition C[rs1] (1024-bit)  $\rightarrow$  100-bit slices
           Route slices to worker EPU's based on EPU_ID
Flags:     None
Exceptions: ContextOverflow

```

CTX.AGG - Context Aggregation

Format: CTX.AGG rd, rs1
Encoding: [0011011][rd][001][rs1][00000][0000000]
Operation: Aggregate 10 worker outputs → manager representation
 $C[rd] \leftarrow f_aggregate(C[rs1])$ where $f \in \{OR, AND, MAJORITY, WEIGHT\}$
Flags: None
Exceptions: None

CTX.SYNTH - Context Synthesis

Format: CTX.SYNTH rd, rs1
Encoding: [0011011][rd][010][rs1][00000][0000000]
Operation: Synthesize 10 manager outputs → queen binary decision
 $X[rd] \leftarrow f_decision(C[rs1])$ where f is Boolean function
Flags: None
Exceptions: None

4.5 Hierarchy Management

HIER.SPAWN - Spawn Agent

Format: HIER.SPAWN rd, rs1, imm5
Encoding: [0100011][rd][000][rs1][imm5][0000000]
Operation: Create new agent at hierarchy level imm5
Inherit from parent agent M[rs1]
Allocate agent ID → X[rd]
Flags: None
Exceptions: ResourceExhausted, HierarchyViolation

HIER.TERM - Terminate Agent

Format: HIER.TERM rs1
Encoding: [0100011][00000][001][rs1][00000][0000000]
Operation: Terminate agent X[rs1], reclaim resources
Update population moments
Flags: None
Exceptions: InvalidAgentID

HIER.EVOL - Evolve Capability

Format: HIER.EVOL rd, rs1, rs2
Encoding: [0100011][rd][010][rs1][rs2][0000000]
Operation: $M[rd] \leftarrow M[rs1] + \alpha \cdot (M[rs2] - M[rs1])$
 where $\alpha = 0.001/\text{cycle learning rate}$
Flags: None
Exceptions: None

4.6 Safety & Constraint Instructions

CONS.CHK - Conservation Check

Format: CONS.CHK rd, rs1, rs2, tol
Encoding: [0101011][rd][000][rs1][rs2][tol]
Operation: $X[rd] \leftarrow 1$ if $|\Sigma \text{inputs} - \Sigma \text{outputs}| < 10^{-\text{tol}/16}$, else 0
 Inputs at M[rs1], outputs at M[rs2]
Flags: STATUS.CON[CONSERVATION] $\leftarrow \sim X[rd]$
Exceptions: None

RES.BUDG - Resource Budget Enforcement

Format: RES.BUDG rd, rs1
Encoding: [0101011][rd][001][rs1][00000][0000000]
Operation: $X[rd] \leftarrow 1$ if $\text{allocation} + \text{cost}(M[rs1]) \leq \text{budget}$, else 0
Flags: STATUS.CON[BUDGET] $\leftarrow \sim X[rd]$
Exceptions: None

BYZ.CONS - Byzantine Consensus

Format: BYZ.CONS rd, rs1, quorum
Encoding: [0101011][rd][010][rs1][quorum][0000000]
Operation: Execute 3-phase Byzantine consensus
 Transaction at address X[rs1], quorum size = quorum
 Result (committed=1, aborted=0) $\rightarrow X[rd]$
Flags: None
Exceptions: ConsensusTimeout, QuorumUnreachable

TMR.VOTE - Triple Modular Redundancy Vote

Format: TMR.VOTE rd, rs1, rs2, rs3
Encoding: [0101011][rd][011][rs1][rs2][rs3]
Operation: $X[rd] \leftarrow \text{majority}(X[rs1], X[rs2], X[rs3])$
Flags: None
Exceptions: None

SAFE.ROLL - Safety Rollback

Format: SAFE.ROLL rs1
Encoding: [0101011][00000][100][rs1][00000][0000000]
Operation: Rollback to checkpoint at address $X[rs1]$
Restore state from Write-Ahead Log
Flags: STATUS.MODE $\leftarrow$ ROLLBACK
Exceptions: CheckpointNotFound, LogCorrupted

4.7 Variational Mechanics

LEVAL - Evaluate Lagrangian

Format: LEVAL rd, rs1, rs2
Encoding: [0111011][rd][000][rs1][rs2][0000000]
Operation: $F[rd] \leftarrow L(q, \dot{q}) = T(\dot{q}) - V(q)$
where $q = M[rs1]$, $\dot{q} = M[rs2]$
Flags: None
Exceptions: None

LGRAD_Q - Gradient w.r.t. Coordinates

Format: LGRAD_Q rd, rs1, rs2
Encoding: [0111011][rd][001][rs1][rs2][0000000]
Operation: $M[rd] \leftarrow \partial L / \partial q$ at $(q=M[rs1], \dot{q}=M[rs2])$
Flags: None
Exceptions: None

LGRAD_V - Gradient w.r.t. Velocities

Format: LGRAD_V rd, rs1, rs2
 Encoding: [0111011][rd][010][rs1][rs2][0000000]
 Operation: $M[rd] \leftarrow \partial L / \partial \dot{q}$ at $(q=M[rs1], \dot{q}=M[rs2])$
 Flags: None
 Exceptions: None

ACTION - Action Integral

Format: ACTION rd, rs1, rs2, imm
 Encoding: [0111011][rd][011][rs1][rs2][imm]
 Operation: $F[rd] \leftarrow \int_{t_1}^{t_2} L dt$
 where $t_1 = F[rs1], t_2 = F[rs2]$
 Integration method: 0=trapezoidal, 1=Simpson
 Flags: None
 Exceptions: IntegrationError

VERLET - Velocity Verlet Step

Format: VERLET rd, rs1, rs2, rs3, rs4
 Encoding: [0111011][rd][100][rs1][rs2][rs3:rs4]
 Operation: $M[rd] \leftarrow 2 \cdot M[rs1] - M[rs2] + M^{-1}[rs3] \cdot F \cdot \Delta t^2$
 where $\Delta t = F[rs4]$
 Flags: None
 Exceptions: None

HAMILT - Hamiltonian Evaluation

Format: HAMILT rd, rs1, rs2, rs3, rs4
 Encoding: [0111011][rd][101][rs1][rs2][rs3:rs4]
 Operation: $F[rd] \leftarrow H = \frac{1}{2} \dot{q}^T \cdot M_0 \cdot \dot{q} + \frac{1}{2} q^T \cdot K_0 \cdot q$
 where $q=M[rs1], \dot{q}=M[rs2], M_0=M[rs3], K_0=M[rs4]$
 Flags: None
 Exceptions: None

AGRAD - Action Gradient Magnitude

Format: AGRAD rd, rs1, rs2
 Encoding: [0111011][rd][110][rs1][rs2][0000000]

Operation: $F[rd] \leftarrow \|\nabla_q S\| = \sqrt{\sum_i (\partial S / \partial q_i)^2}$
Trajectory at $M[rs1]$, weights at $M[rs2]$
Flags: None
Exceptions: None

ATHRESH - Action Gradient Threshold

Format: ATHRESH rd, rs1, rs2, imm
Encoding: $[0111011][rd][111][rs1][rs2][imm]$
Operation: $X[rd] \leftarrow (\|\nabla_q S\| / \sigma^2_q > 10^{\{-imm/32\}}) ? 1 : 0$
where $\nabla_q S = M[rs1]$, $\sigma^2_q = F[rs2]$
Flags: STATUS.MODE[ACTIVE] $\leftarrow X[rd]$
Exceptions: None

4.8 System Instructions

ECALL - Environment Call

Format: ECALL
Encoding: $[1110011][00000][000][00000][000000000000]$
Operation: Request service from execution environment
Service number in $X[10]$, arguments in $X[11]-X[17]$
Return value in $X[10]$
Flags: None
Exceptions: EnvironmentCall (trap to supervisor)

ERET - Exception Return

Format: ERET
Encoding: $[1110011][00000][001][00000][000000000000]$
Operation: Return from exception handler
PC $\leftarrow$ EPC, STATUS $\leftarrow$ restored from exception stack
Flags: STATUS restored
Exceptions: None

MRET - Machine Mode Return

Format: MRET
Encoding: [1110011][00000][010][00000][000000000000]
Operation: Return from machine-mode trap
PC $\leftarrow$ MEPC, STATUS restored from mstatus
Flags: Machine status restored
Exceptions: None

WFI - Wait for Interrupt

Format: WFI
Encoding: [1110011][00000][011][00000][000000000000]
Operation: Wait for interrupt, enter low-power state
Resume on interrupt or event broadcast
Flags: STATUS.MODE $\leftarrow$ HIBERNATED
Exceptions: None

CSR Instructions

CSRRW rd, csr, rs1: Atomic swap: $t = \text{CSR}[csr]; \text{CSR}[csr] = X[rs1]; X[r$
CSRRS rd, csr, rs1: Atomic read/set bits: $t = \text{CSR}[csr]; \text{CSR}[csr] = t$
CSRRC rd, csr, rs1: Atomic read/clear bits: $t = \text{CSR}[csr]; \text{CSR}[csr] =$
CSRRWI, CSRRSI, CSRRCI: Immediate versions (5-bit immediate)



5. MEMORY MODEL

5.1 Consistency Model

- Release Consistency: Operations separated by synchronization points
- Event-based ordering: Event registers provide explicit synchronization
- Write-Ahead Logging: All state changes logged before commitment

5.2 Atomic Operations

AMOSWAP.W: Atomic swap word (32-bit)
AMOADD.D: Atomic add doubleword (64-bit)
AMOAND.D: Atomic AND doubleword
AMOOR.D: Atomic OR doubleword
AMOXOR.D: Atomic XOR doubleword
AMOMAX.D: Atomic maximum
AMOMIN.D: Atomic minimum

5.3 Memory Ordering

- **Load-Load:** Ordered within same EPU, relaxed across EPUs
- **Load-Store:** Ordered within same EPU
- **Store-Store:** Ordered to same address, relaxed otherwise
- **Event synchronization:** ESET/ECLEAR provide release/acquire semantics

5.4 Alignment Requirements

Data Size	Required Alignment
Byte	1 byte
Halfword	2 bytes
Word	4 bytes
Doubleword	8 bytes
Quadword	16 bytes
Context	128 bytes

Misaligned access generates alignment exception (trap code 4).

6. EXCEPTION AND INTERRUPT ARCHITECTURE

6.1 Exception Types

6.1.1 Synchronous Exceptions (Traps)

Trap Code	Cause	Description
0	InstructionMisaligned	PC not 4-byte aligned
1	InstructionAccessFault	Instruction fetch failed
2	IllegalInstruction	Invalid opcode or encoding
3	Breakpoint	EBREAK instruction
4	LoadAddressMisaligned	Load address misaligned
5	LoadAccessFault	Load failed
6	StoreAddressMisaligned	Store address misaligned
7	StoreAccessFault	Store failed
8	EnvironmentCall	ECALL instruction
9	ConstraintViolation	Moment/Hausdorff violation
10	ResourceExhausted	No resources for agent spawn
11	ConsensusTimeout	Byzantine consensus timeout
12	EventOutOfRange	Event register index > 511
13	HierarchyViolation	Invalid hierarchy operation

6.1.2 Asynchronous Interrupts

Interrupt	Priority	Description
0	7 (highest)	Machine timer interrupt
1	6	Machine external interrupt
2	5	Supervisor timer interrupt
3	4	Supervisor external interrupt

Interrupt	Priority	Description
4	3	User timer interrupt
5	2	User external interrupt
6	1	Event broadcast interrupt
7	0	Performance monitor interrupt

6.2 Trap Handling

6.2.1 Trap Registers

mtvec: Machine trap vector base address
 mepc: Machine exception PC (PC at trap)
 mcause: Machine trap cause (exception code)
 mtval: Machine trap value (additional info)
 mstatus: Machine status register
 mscratch: Machine scratch register

6.2.2 Trap Entry

1. **mcause** $\leftarrow$ trap cause
2. **mepc** $\leftarrow$ PC of trapped instruction
3. **mtval** $\leftarrow$ faulting address or instruction
4. **PC** $\leftarrow$ mtvec (direct mode) or mtvec + 4×mcause (vectored)
5. **mstatus.MPIE** $\leftarrow$ mstatus.MIE (save interrupt enable)
6. **mstatus.MIE** $\leftarrow$ 0 (disable interrupts)
7. **mstatus.MPP** $\leftarrow$ current privilege mode

6.2.3 Trap Return

1. **PC** $\leftarrow$ mepc
2. **mstatus.MIE** $\leftarrow$ mstatus.MPIE (restore interrupt enable)
3. **Privilege mode** $\leftarrow$ mstatus.MPP

6.3 Interrupt Vector Table

Located at address in mtvec (must be 256-byte aligned).

Offset	Handler
0x000:	Synchronous exception handler
0x040:	Machine timer interrupt handler
0x080:	Machine external interrupt handler
0x0C0:	Supervisor timer interrupt handler
0x100:	Supervisor external interrupt handler
0x140:	Event broadcast interrupt handler
0x180:	Performance monitor interrupt handler

7. PRIVILEGED ARCHITECTURE

7.1 Privilege Levels

Level	Encoding	Name	Description
0	00	User	Application code
1	01	Supervisor	Operating system
2	10	(Reserved)	
3	11	Machine	Firmware, hypervisor

7.2 Control and Status Registers (CSRs)

7.2.1 Machine-Level CSRs

Address	Name	Description
0x300	mstatus	Machine status register
0x301	misa	ISA and extensions
0x304	mie	Machine interrupt enable
0x305	mtvec	Machine trap handler base

Address	Name	Description
0x340	mscratch	Machine scratch register
0x341	mepc	Machine exception PC
0x342	mcause	Machine trap cause
0x343	mtval	Machine bad address or instr
0x344	mip	Machine interrupt pending
0x3A0	mcycle	Cycle counter
0x3B0	mcycleh	Upper 32 bits of cycle counter

7.2.2 Supervisor-Level CSRs

Address	Name	Description
0x100	sstatus	Supervisor status
0x104	sie	Supervisor interrupt enable
0x105	stvec	Supervisor trap handler
0x140	sscratch	Supervisor scratch
0x141	sepc	Supervisor exception PC
0x142	scause	Supervisor trap cause
0x143	stval	Supervisor bad address
0x144	sip	Supervisor interrupt pending
0x180	satp	Supervisor address translation

7.2.3 PICAPD-Specific CSRs

Address	Name	Description
0x800	mevtmask	Event register access mask
0x801	mmoment	Moment register configuration

Address	Name	Description
0x802	mcontext	Context flow control
0x803	magmcfg	AGM unit configuration
0x804	mhierarchy	Hierarchy level control
0x805	mtrustcfg	Trust score configuration
0x806	mvariational	Variational mechanics control
0x807	msafety	Safety constraint configuration

7.3 Memory Protection

7.3.1 Physical Memory Protection (PMP)

8 PMP registers control access to physical memory regions:

`pmpcfg0-7`: Configuration (R/W/X, lock, mode)

`pmpaddr0-7`: Address registers (22-bit for 34-bit physical)

7.3.2 Virtual Memory (Sv39)

- 39-bit virtual address, 34-bit physical address
- 4KB page size, three-level page table
- ASID: 9-bit Address Space Identifier
- TLB: 64-entry fully-associative

7.4 I/O Protection

- **Memory-mapped I/O**: Protected via PMP
- **Direct I/O instructions**: Privileged (machine mode only)
- **DMA**: Controlled via IOMMU with separate page tables

8. ASSEMBLY SYNTAX

8.1 Instruction Syntax

<instruction> <dest>, <src1>, <src2>, <immediate>

Examples:

```
ESET x5, 42          # Set event register 42, broadcast mask in x5
MOM.CALC f10, x6, 2   # Calculate  $\mu_2$  from distribution at [x6]
AGM.ITER a3, a1, a2   # AGM iteration:  $a_3 = (a_1.a + a_2.g)/2, \sqrt{a_1.a \times a_2.g}$ 
CONS.CHK x7, m8, m9, 32 # Check conservation with tolerance  $10^{-2}$ 
```



8.2 Register Naming Conventions

Integer registers:	x0-x31	(x0 is hardwired zero)
Floating-point:	f0-f31	
Event registers:	e0-e511	(bit-addressable)
Moment registers:	m0-m127	(grouped as m0-m3, m4-m7, ...)
Context registers:	c0-c63	
AGM registers:	a0-a31	
Vector clocks:	v0-v15	
Trust scores:	t0-t255	

8.3 Pseudo-instructions

NOP:	ADDI x0, x0, 0
LI rd, imm:	Load immediate (expands to LUI + ADDI)
LA rd, symbol:	Load address (PC-relative)
MV rd, rs:	ADDI rd, rs, 0
NOT rd, rs:	XORI rd, rs, -1
SEQZ rd, rs:	SLTIU rd, rs, 1
SNEZ rd, rs:	SLTU rd, x0, rs

8.4 Directives

.data	Switch to data section
.text	Switch to text section
.align n	Align to 2^n bytes
.word w1, w2, ...	Emit 32-bit words

.dword d1, d2, ... Emit 64-bit doublewords
.ascii "str" Emit ASCII string
.asciz "str" Emit null-terminated ASCII string
.equ name, value Define constant

APPENDIX A: INSTRUCTION ENCODING REFERENCE

A.1 Complete Opcode Map

Opcode [6:0]	funct3	Instruction	Type	Operation
0000011	000	ESET	E	Set event register
0000011	001	ECLEAR	E	Clear event register
0000011	010	ETEST	E	Test event state
0000011	011	EWAIT	E	Wait for event
0000011	100	EBCAST	E	Broadcast event
0001011	000	MOM.CALC	M	Calculate moment
0001011	001	MOM.COMP	M	Compress moments
0001011	010	MOM.XPORT	M	Moment transport
0001011	011	MOM.REAL	M	Realizability check
0010011	000	AGM.ITER	A	AGM iteration
0010011	001	ELI.COMP	A	Elliptic integral
0010011	010	TXF.PASS	A	Transfer function passivity
0011011	000	CTX.SLICE	C	Context slicing
0011011	001	CTX.AGG	C	Context aggregation

Opcode [6:0]	funct3	Instruction	Type	Operation
0011011	010	CTX.SYNTH	C	Context synthesis
0100011	000	HIER.SPAWN	H	Spawn agent
0100011	001	HIER.TERM	H	Terminate agent
0100011	010	HIER.EVOL	H	Evolve capability
0101011	000	CONS.CHK	S	Conservation check
0101011	001	RES.BUDG	S	Resource budget
0101011	010	BYZ.CONNS	S	Byzantine consensus
0101011	011	TMR.VOTE	S	Triple modular redundancy
0101011	100	SAFE.ROLL	S	Safety rollback
0111011	000	LEVAL	V	Evaluate Lagrangian
0111011	001	LGRAD_Q	V	Gradient w.r.t. q
0111011	010	LGRAD_V	V	Gradient w.r.t. $\dot{q}$
0111011	011	ACTION	V	Action integral
0111011	100	VERLET	V	Verlet integration
0111011	101	HAMILT	V	Hamiltonian
0111011	110	AGRAD	V	Action gradient
0111011	111	ATHRESH	V	Action threshold

A.2 Immediate Field Encodings

imm12 (12-bit signed):

Bits 31:20 in instruction
 Sign-extended to 64 bits for arithmetic
 Range: -2048 to 2047

imm5 (5-bit unsigned):

Bits 11:7 in Type-M instructions

Range: 0-31 (for moment order k)

tol (7-bit tolerance):

Bits 6:0 in Type-S instructions

Encodes tolerance as $10^{\{-tol/16\}}$

tol=0: $10^{-0} = 1$ (disabled)

tol=32: $10^{-2} = 0.01$

tol=64: $10^{-4} = 0.0001$

tol=96: $10^{-6} = 0.000001$

tol=127: $10^{-7.94} \approx 1.28 \times 10^{-8}$ (strictest)

END OF PICAPD ISA SPECIFICATION v1.0

This document defines the complete programmer-visible architecture of the PICAPD instruction set. It provides the contract between hardware and software, enabling correct assembly programming and compiler implementation without ambiguity about instruction behavior or architectural state.